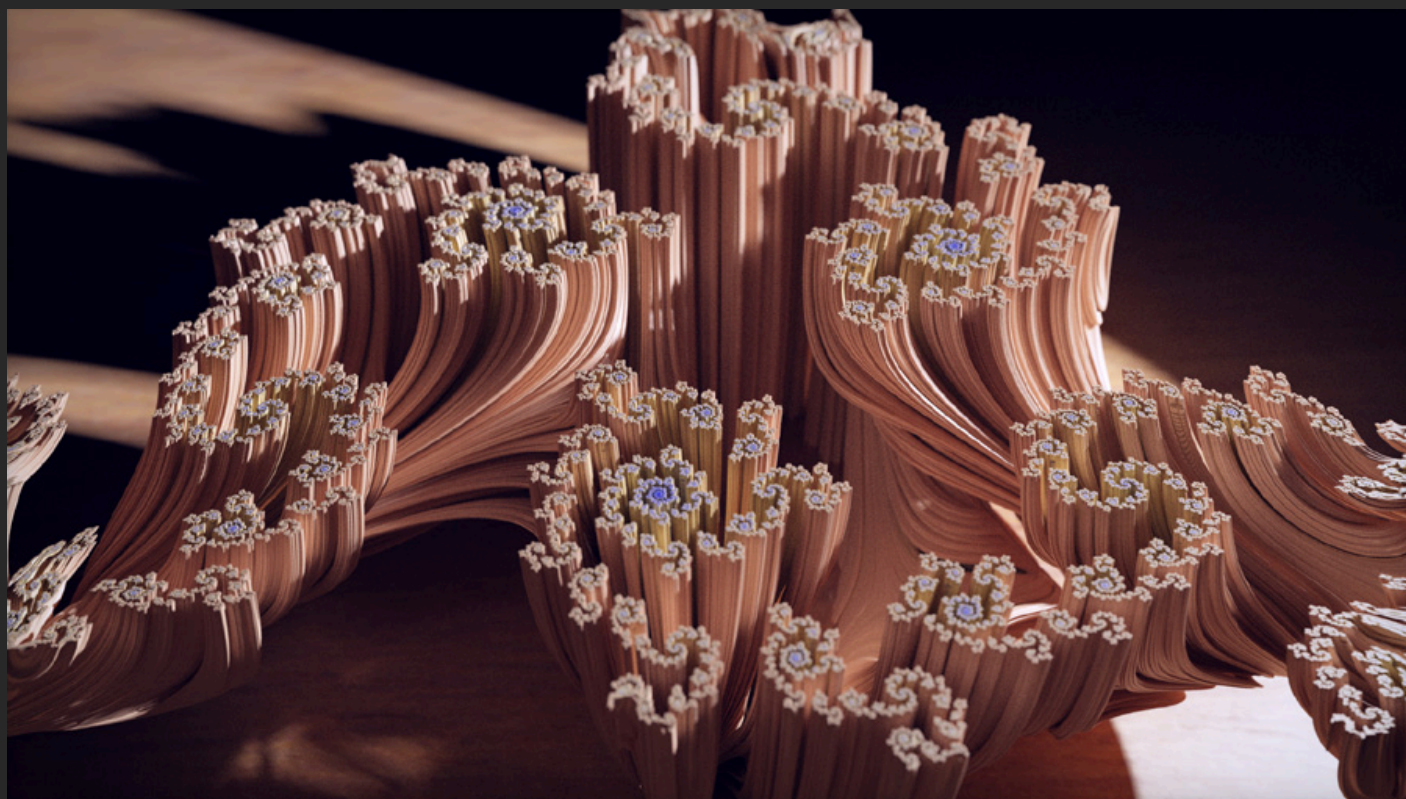


Inigo Quilez :: articles :: 3D Julia sets - 2001



When I first saw images of quaternionic Julia back in 1999 the topic was already a decade old in the literature (check [this paper by Hart, Sandin, Kauffman](#)). But like many aficionados of fractals and 3D rendering, I gave it a go of my own. In the process I reinvented the wheel somehow, by defining my own hacky 3D number system which eventually boiled down to be a super simplified and special case of quaternions. I think reinventing the wheel is really an important part of learning. Some other things about rendering Julia sets I learnt from [Keenan Crane](#), and some other things I learnt or discovered on my own. I'll write about those mostly here. Although I consider this article a complement to my article on [computing the distance to a Julia set](#), which I think you should read first. Anyways, for this one I'll assume you are already more or less familiar with the maths of rendering complex Julia and Mandelbrot sets, and with raymarching.



A raymarched Julia set, cut in half ([video](#))

Intersection

So, unlike rendering in the complex plane (in 2D) which is easy and direct, when rendering quaternionic (4D) fractals in 3D we have to give up some nice things. First, since we are going to be rendering in 3D, we need to decide which three of the four components of the quaternion plane H we'll be using for display purposes. This is an easy decision - just pick any three. We could use some fancy perspective or stereographics projection and use the four of them, but for now we can make our lives easier and just pick three.

The real loss when rendering in 3D is that (not always, but generally) we are going to be rendering only the edge/surface/border of the filled Julia set, or in other words, the actual real Julia set. We won't have fancy exterior coloring, and we will likely see very little of the inner parts of the set, since in 3D we cannot see through surfaces. Of course clipping planes can help us explore the interior, but let's ignore that for now.

So, our problem is now that of raycasting in search for an intersection point between 3D rays and our Julia set. As with all raycasting of shapes with no direct closed form intersection function, we can employ **raymarching of SDFs**. This is a powerful technique, provided we have an SDF, ie, a Signed Distance Function, to the object we are trying to raycast. Naturally you can perform an **analytic intersection test** first with the fractal's bounding box or sphere to quickly discard rays that miss the fractal, but also to speed things up by limiting the raymarching to the segment between the entry and exit points of the ray inside the bounding volume.

Luckily for us, Julia set of polynomial equations, including the classic $f(q) = q^2 + c$ of course, do have a simple way to compute **the distance** to a point (provided we aren't too far from it), namely:

$$d(q_0) = \frac{\nabla G(q_0)}{|\nabla(q_0)|} = \frac{|q_n| \cdot \log|q_n|}{|q'_n|}$$

The derivation of this SDF itself is based on the Boettcher mapping of the exterior of the set and its Green function or Hubbard-Douady potential. Read the article to learn more about that part.

Once the intersection has been found between the ray and the fractal, it's time to compute a surface normal for lighting purposes. Knowing that our shape is a fractal it is a bit weird to talk about surface normals, since the surface is generally not differentiable (nor smooth). But, in the end of the day we are setting a resolution or threshold or sampling rate that is effectively band limiting the detail of the shape, so a normal can be effectively computed.

Surface normals, method 1 : gradient of $d(q_0)$

The simplest way to get a normal is through **the gradient of the SDF**. This is our typical numerical differentiation method applied directly to the SDF $d(q_0)$. We know this works and not much can go wrong with it. An implementation of this method for a traditional Julia set $f(q)=q^2+c$ is below and also at <https://www.shadertoy.com/view/MsfGRr> as METHOD 0:

```
vec3 calcNormal( in vec3 q, in vec4 c )
{
    const vec2 e = vec2(0.001,0.0);
    vec4 qa=vec4(q+e.xyy,0.0); float mq2a=length2(qa), md2a=1.0;
    vec4 qb=vec4(q-e.xyy,0.0); float mq2b=length2(qb), md2b=1.0;
    vec4 qc=vec4(q+e.yxy,0.0); float mq2c=length2(qc), md2c=1.0;
    vec4 qd=vec4(q-e.yxy,0.0); float mq2d=length2(qd), md2d=1.0;
    vec4 qe=vec4(q+e.yyx,0.0); float mq2e=length2(qe), md2e=1.0;
    vec4 qf=vec4(q-e.yyx,0.0); float mq2f=length2(qf), md2f=1.0;
    for( int i=0; i<numIterations; i++ )
    {
        md2a *= mq2a; qa = qsqr(qa) + c; mq2a = length2(qa);
        md2b *= mq2b; qb = qsqr(qb) + c; mq2b = length2(qb);
        md2c *= mq2c; qc = qsqr(qc) + c; mq2c = length2(qc);
        md2d *= mq2d; qd = qsqr(qd) + c; mq2d = length2(qd);
        md2e *= mq2e; qe = qsqr(qe) + c; mq2e = length2(qe);
        md2f *= mq2f; qf = qsqr(qf) + c; mq2f = length2(qf);
    }
    float da = sqrt(mq2a/md2a)*log2(mq2a);
    float db = sqrt(mq2b/md2b)*log2(mq2b);
    float dc = sqrt(mq2c/md2c)*log2(mq2c);
    float dd = sqrt(mq2d/md2d)*log2(mq2d);
    float de = sqrt(mq2e/md2e)*log2(mq2e);
    float df = sqrt(mq2f/md2f)*log2(mq2f);

    return normalize( vec3(da-db,dc-dd,de-df) );
}
```

But maybe we can do better.

Surface normals, method 2 : gradient of $G(q_0)$

Instead of evaluating the gradient of $d(q_0)$ numerically, we can try doing it analytically. That results in the following (removing the dependency on q_0 now for clarity):

$$\nabla d = \frac{\nabla G \cdot |\nabla G| - G \cdot \nabla |\nabla G|}{|\nabla G|^2}$$

which is a bit of a monster. However we are only interested in evaluating it at the surface of the set, where we know that $G=0$. There, the expression simplifies to

$$\nabla d = \frac{\nabla G}{|\nabla G|}$$

Or in other words, we can compute the surface normal as the normalize gradient of the Douady-Hubbard potential, $G(q_0)$. If anything, we can compute that one numerically. An implementation of this method for a traditional Julia set $f(q)=q^2+c$ is below and also at <https://www.shadertoy.com/view/MsfGRr> as METHOD 1:

```
vec3 calcNormal( in vec3 q, in vec4 c )
{
    const vec2 e = vec2(0.001,0.0);
    vec4 qa = vec4(q+e.xyy,0.0);
    vec4 qb = vec4(q-e.xyy,0.0);
    vec4 qc = vec4(q+e.yxy,0.0);
    vec4 qd = vec4(q-e.yxy,0.0);
    vec4 qe = vec4(q+e.yyx,0.0);
    vec4 qf = vec4(q-e.yyx,0.0);

    for( int i=0; i<numIterations; i++ )
    {
        qa = qsqr(qa) + c;
        qb = qsqr(qb) + c;
        qc = qsqr(qc) + c;
        qd = qsqr(qd) + c;
        qe = qsqr(qe) + c;
        qf = qsqr(qf) + c;
    }
    return normalize( vec3(log2(qlength2(qa))-log2(qlength2(qb)),
                          log2(qlength2(qc))-log2(qlength2(qd)),
                          log2(qlength2(qe))-log2(qlength2(qf))) );
}
```

This is already more efficient. But perhaps we can do even better.

Surface normals, method 3 : analytical gradient of $G(q_0)$

The idea now is consider that $G(q_0)$ is actually

$$G(q_0) = \lim_{n \rightarrow \infty} \frac{1}{p^n} \cdot \log|q_n|$$

where q_n is of course a huge polynomial of q_0 of degree p_n . If we split a quaternion into its four components $q = x + i \cdot y + j \cdot z + k \cdot w$ and we take the derivatives of G with respect to the four coordinates of our space q_0 , then

$$\nabla G(q_0) = \begin{pmatrix} \frac{\partial G}{\partial x_0} \\ \frac{\partial G}{\partial y_0} \\ \frac{\partial G}{\partial z_0} \\ \frac{\partial G}{\partial w_0} \end{pmatrix} = \frac{1}{p^n \cdot |q_n|^2} \begin{pmatrix} \frac{\partial x_n}{\partial x_0} & \frac{\partial y_n}{\partial x_0} & \frac{\partial z_n}{\partial x_0} & \frac{\partial w_n}{\partial x_0} \\ \frac{\partial x_n}{\partial y_0} & \frac{\partial y_n}{\partial y_0} & \frac{\partial z_n}{\partial y_0} & \frac{\partial w_n}{\partial y_0} \\ \frac{\partial x_n}{\partial z_0} & \frac{\partial y_n}{\partial z_0} & \frac{\partial z_n}{\partial z_0} & \frac{\partial w_n}{\partial z_0} \\ \frac{\partial x_n}{\partial w_0} & \frac{\partial y_n}{\partial w_0} & \frac{\partial z_n}{\partial w_0} & \frac{\partial w_n}{\partial w_0} \end{pmatrix} \cdot \begin{pmatrix} x_n \\ y_n \\ z_n \\ w_n \end{pmatrix}$$

or in short:

$$\nabla G(q_0) = \frac{1}{p^n \cdot |q_n|^2} J(q_n) \cdot q_n$$

Now, the fraction in front of the matrix we can ignore it, since we'll need to normalize our gradient vector anyways in order to perform lighting. The big matrix in the middle is the Jacobian of q_n with respect to q_0 , which we can compute iteratively together with q_n itself, though the chain rule. For example, for a traditional Julia set of $f(q) = q^2 + c$, we'd get

$$J(q_{k+1}) = 2 \begin{pmatrix} x_k & -y_k & -z_k & -w_k \\ y_k & x_k & 0 & 0 \\ z_k & 0 & x_k & 0 \\ w_k & 0 & 0 & x_k \end{pmatrix} \cdot J(q_k)$$

with $J_0 = I$, the identity matrix. Note that the 2 in front of the matrix can be discarded for the computation as well since the final gradient will be normalized afterwards. The code for this method can be found again at <https://www.shadertoy.com/view/MsfGRr> under METHOD 2:

```
vec3 calcNormal( in vec3 q0, in vec4 c )
{
    vec4 q = vec4(q0,0.0);
    mat4x4 J = identity_4x4();
    for( int i=0; i<numIterations; i++ )
    {
        J = J*mat4x4(q.x, -q.y, -q.z, -q.w,
                    q.y,  q.x,  0.0,  0.0,
                    q.z,  0.0,  q.x,  0.0,
                    q.w,  0.0,  0.0,  q.x);
        q = qsqr(q) + c;
    }
    return normalize( (J*q).xyz );
}
```

Now, something beautiful happens when the above formula is specialized to complex numbers (for regular 2D Julia sets): the Jacobian matrix, which is a 2x2 matrix, becomes of the form

$$J(q_{k+1}) = 2 \begin{pmatrix} x_k & -y_k \\ y_k & x_k \end{pmatrix} \cdot J(q_k)$$

and given that the **Cauchy-Riemann equations** of holomorphic functions, it means that we can just keep track of the complex derivative of q_n with respect to q_0 and get rid of the Jacobians, which makes it twice as fast:

$$\nabla G(z_0) = \frac{1}{p^n \cdot |z_n|^2} \cdot z_n \cdot (z'_n)^*$$

or in code,

```
vec3 calcNormal( in vec3 zo, in vec4 c )
{
    vec4 z = vec4(zo,0.0);
    vec2 d = vec2(1.0,0.0);
    for( int i=0; i<numIterations; i++ )
    {
        d = cmul(z,d);
    }
}
```

```

    z = csqr(z) + c;
}
return normalize( cmul(z, cconj(dz)) );
}

```

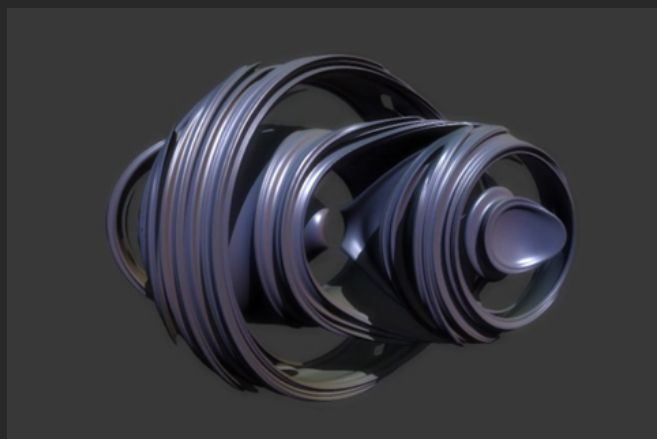
Unfortunately, although the Jacobian for the quaternion version also has lots of symmetries in it, I haven't been able to simplify it to an equivalent $q_n \cdot q_n^*$ form. I think this is due to the fact that I believe the equivalent of the Cauchy-Riemann equations for quaternions basically don't exist or don't work in that they make the meaning of a derivative useless.



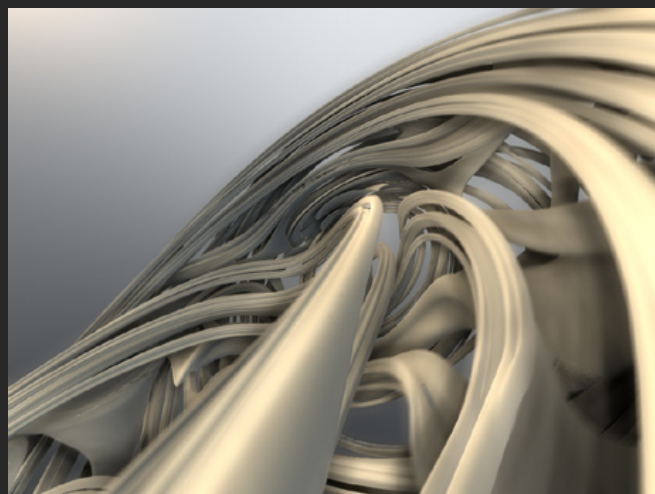
Raymarched Julia set - 2001



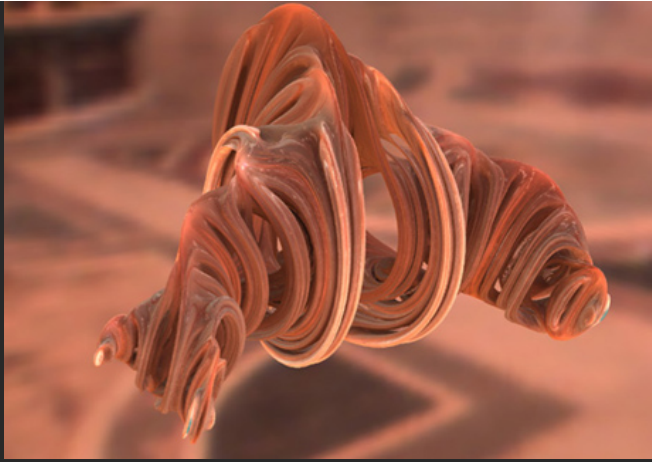
A raymarched Julia set - 2005 (hover to play)



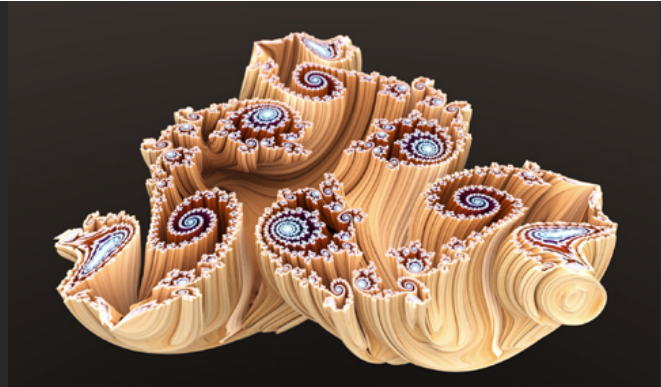
Raymarched cubic Julia set - 2005



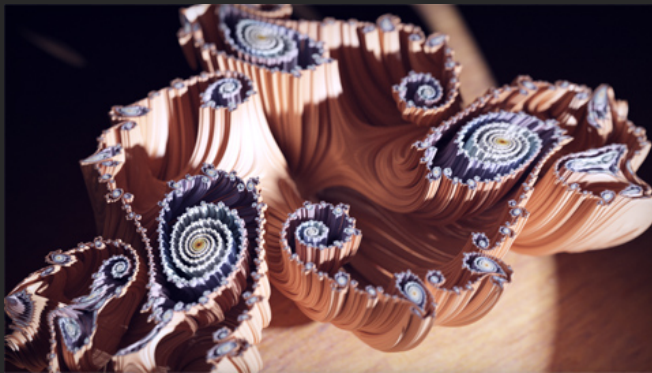
Realtime raymarched Julia set - 2007 (hover to play)



Raymarched cubic Julia set (realtime) - 2013

[\(code\)](#)

Raymarched cubic Julia set (realtime)- 2020

[\(code\)](#)

Raymarched cubic Julia set - 2020



Raymarched cubic Julia set - 2020

[inigo quilez](#) - learning computer graphics since 1994